



ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO
FACULTAD: INFORMÁTICA Y ELECTRÓNICA
ESCUELA DE INGENIERÍA EN SISTEMAS
CARRERA: SOFTWARE

GUÍA DE LABORATORIO DE APLICACIONES INFORMÁTICAS II

PRÁCTICA No. 3

1. DATOS GENERALES:

NOMBRE DEL ESTUDIANTE

Stiven Apolo

CODIGO DEL ESTUDIANTE

7243

FECHA DE REALIZACIÓN:

2026/07/07

FECHA DE ENTREGA:

2026/07/07

2. OBJETIVO(S):

2.1. GENERAL

Aplicar el proceso de mantenimiento de software sobre la aplicación ClinicFlow_corrected, registrando los módulos intervenidos, priorizando los problemas identificados, describiendo los cambios implementados, verificando los resultados mediante pruebas y definiendo el procedimiento de despliegue del sistema modificado.

2.2. ESPECÍFICOS

- Registrar las áreas funcionales y técnicas de la aplicación que fueron objeto de mantenimiento.
- Analizar y priorizar los problemas según criterios de severidad, impacto y urgencia.
- Describir los cambios implementados y las buenas prácticas aplicadas durante el mantenimiento.
- Definir las pruebas realizadas y los criterios de aceptación utilizados para validar los cambios.
- Documentar el procedimiento de despliegue del sistema modificado en entorno local mediante Docker.

3. METODOLOGÍA

La práctica se desarrolló mediante un enfoque de mantenimiento incremental. Primero se revisaron los problemas identificados en la aplicación ClinicFlow original. Luego se analizaron los módulos afectados en el repositorio mantenido ClinicFlow_corrected. Posteriormente se verificaron los archivos modificados, los scripts agregados, la configuración del entorno, los ajustes de frontend y backend, y las mejoras de experiencia de usuario. Finalmente se definieron evidencias mediante capturas, pruebas funcionales y un procedimiento de despliegue reproducible.

Las técnicas aplicadas fueron revisión estática de código, análisis de configuración, priorización impacto-esfuerzo, verificación funcional manual y validación de despliegue local con Docker Compose.

4. EQUIPOS Y MATERIALES:

- Computador
- Entorno integrado de desarrollo (IDE)
- Aula virtual
- Acceso a internet
- Repositorio GitHub: StivenApolo/ClinicFlow_corrected
- Docker Desktop y Docker Compose
- Node.js 18 o superior
- MongoDB, Redis y RedisInsight
- Bibliografía

5. MARCO TEÓRICO:

El mantenimiento de software comprende el conjunto de actividades realizadas sobre una aplicación después de su desarrollo inicial, con el propósito de corregir fallos, adaptar el sistema a nuevos entornos, mejorar sus características funcionales o no funcionales y prevenir defectos futuros. En esta práctica se trabajó con cuatro enfoques principales: mantenimiento correctivo, adaptativo, perfectivo y preventivo.

El mantenimiento correctivo se orienta a corregir errores detectados en la aplicación. El mantenimiento adaptativo permite que el sistema funcione en nuevos entornos, versiones o configuraciones. El mantenimiento perfectivo mejora la usabilidad, rendimiento, experiencia de usuario o calidad interna. El mantenimiento preventivo busca disminuir riesgos futuros mediante refactorización, documentación, actualización controlada y pruebas.

La priorización de problemas se realizó considerando severidad, impacto y urgencia. La severidad permite medir la gravedad técnica del problema; el impacto permite estimar cuánto afecta al usuario o al funcionamiento del sistema; y la urgencia permite definir qué tan pronto debe ser atendido el problema para evitar fallas o bloqueos en el proceso de uso y despliegue.

6. PROCEDIMIENTO:

6.1. Registro de módulos / áreas a intervenir

Se registraron los principales módulos y archivos intervenidos en el proceso de mantenimiento. Estos elementos corresponden a configuración del entorno, datos iniciales, comunicación frontend-backend, manejo de estados, caché Redis, experiencia de usuario, documentación, pruebas y despliegue.

Área / módulo	Archivo relacionado	Intervención realizada
Configuración general	.env.docker, server/.env.example	Variables de entorno para Docker, backend, MongoDB, Redis, CORS y JWT.
Despliegue local	docker-compose.yml, ARRANQUE_LOCAL.md	Orquestación de client, server, mongo, redis y redisinsight.
Datos iniciales	server/scripts/seed-demo.js, server/package.json	Carga de usuarios, departamentos, doctores y citas demo.
Consumo de API	client/src/lib/apiBaseUrl.ts	Centralización de URL del backend para local, Docker o producción.
Comunicación en tiempo real	client/src/socket.tsx, server/src/config/socket.js	Uso de Socket.IO con URL centralizada.
Estado global Redux	client/src/store/slice/adminSlice.ts y demás slices	Ajustes en endpoints, manejo de errores y actualización de estado.
Caché Redis	server/src/controller/*.js y seed-demo.js	Invalidación de claves para evitar datos obsoletos.

Área / módulo	Archivo relacionado	Intervención realizada
Experiencia de usuario	client/src/components/common/DepartmentDesign.tsx, global-error.tsx	Estado vacío, reintento, navegación y pantalla de error global.
Pruebas	package.json, ejecución manual, navegador y Docker	Pruebas de arranque, seed, navegación, API, CORS y verificación visual.

6.2. Análisis y priorización

La priorización se realizó usando tres criterios: severidad, impacto y urgencia. Se dio mayor prioridad a los problemas que impedían levantar la aplicación, consumir la API o demostrar datos funcionales. Los problemas de refactorización y pruebas automatizadas se consideraron importantes, pero con mayor esfuerzo de implementación.

Prioridad	Problema	Severidad	Impacto	Urgencia	Justificación
1	Variables de entorno incompletas o no alineadas	Alta	Alto	Alta	Sin configuración correcta no funcionan backend, base de datos, Redis o JWT.
2	Problema de CORS entre frontend y backend	Alta	Alto	Alta	Impide que el frontend consuma la API.
3	La app no trae datos iniciales para demo	Alta	Alto	Alta	Las vistas aparecen vacías y no se puede validar el flujo funcional.
4	Redirecciones y consumo de API dependen del entorno	Media-Alta	Medio-Alto	Media	Afecta el funcionamiento en local, Docker o producción.
5	Error o inconsistencia en	Alta	Alto	Media	Puede actualizar mal la interfaz después de

Prioridad	Problema	Severidad	Impacto	Urgencia	Justificación
	reducers de Redux				editar datos.
6	Manejo de caché Redis con datos obsoletos	Media-Alta	Medio-Alto	Media	Puede mostrar datos antiguos después de cambios.
7	Falta de mensajes de error y feedback claros	Media	Medio	Media	Afecta la comprensión del usuario.
8	Estado vacío poco informativo	Media	Medio	Media	La interfaz no orienta al usuario cuando no hay departamentos.
9	Falta de pruebas automatizadas	Alta	Alto	Baja-Media	Es importante para prevenir regresiones, pero requiere más tiempo.
10	Controladores grandes y lógica mezclada	Media-Alta	Medio-Alto	Baja-Media	Dificulta mantenimiento futuro; requiere refactorización progresiva.

6.3. Implementación de cambios

Los cambios se implementaron aplicando buenas prácticas de configuración, separación de responsabilidades, centralización de URLs, reutilización de componentes, manejo controlado de errores, uso de variables de entorno y documentación de arranque. También se incorporaron scripts reproducibles para facilitar la demostración del sistema y reducir errores manuales.

Cambio implementado	Descripción	Archivo o módulo	Tipo de mantenimiento
Datos iniciales para demo	Se agregó el script seed:demo y el archivo seed-demo.js para cargar datos base.	server/package.json, server/scripts/seed-demo.js	Correctivo / Preventivo

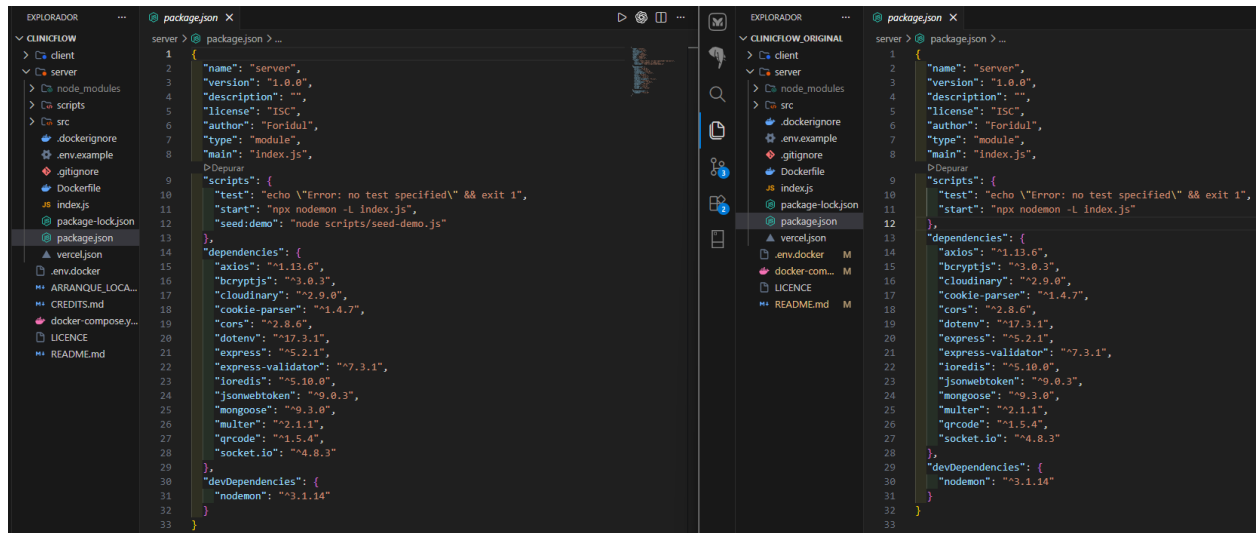
Cambio implementado	Descripción	Archivo o módulo	Tipo de mantenimiento
Variables de entorno	Se completaron variables para CORS, MongoDB, Redis, JWT y revalidación.	.env.docker, server/.env.example	Correctivo / Adaptativo
CORS frontend-backend	El backend mantiene CORS por variable CORS_ORIGIN y credentials: true.	server/src/app.js, .env.docker	Correctivo / Adaptativo
Consumo de API por entorno	Se creó apiBaseUrl.ts para resolver la URL del backend según variables públicas.	client/src/lib/apiBaseUrl.ts	Adaptativo
Socket.IO adaptable	El socket usa API_BASE_URL y evita URL fija.	client/src/socket.tsx	Adaptativo
Redux y endpoints	Los slices usan apiUrl y rejectWithValue para errores controlados.	client/src/store/slice/adminSlice.ts	Correctivo / Perfectivo
Caché Redis	Se invalidan claves de dashboard,	server/src/controller/doctor.controller.js, seed-demo.js	Correctivo / Preventivo

Cambio implementado	Descripción	Archivo o módulo	Tipo de mantenimiento
	appointment y datos demo al modificar el estado.		
Estado vacío de departamentos	Se agregó una vista informativa con acciones de refrescar, volver al inicio y recargar.	client/src/components/common/DepartmentDesign.tsx	Perfectivo
Error global	Se agregó una pantalla global de error con opciones de reintento y retorno al inicio.	client/src/app/global-error.tsx	Perfectivo
Documentación de arranque	Se documentó el levantamiento con Docker y ejecución manual.	README.md, ARRANQUE_LOCAL.md	Preventivo / Adaptativo

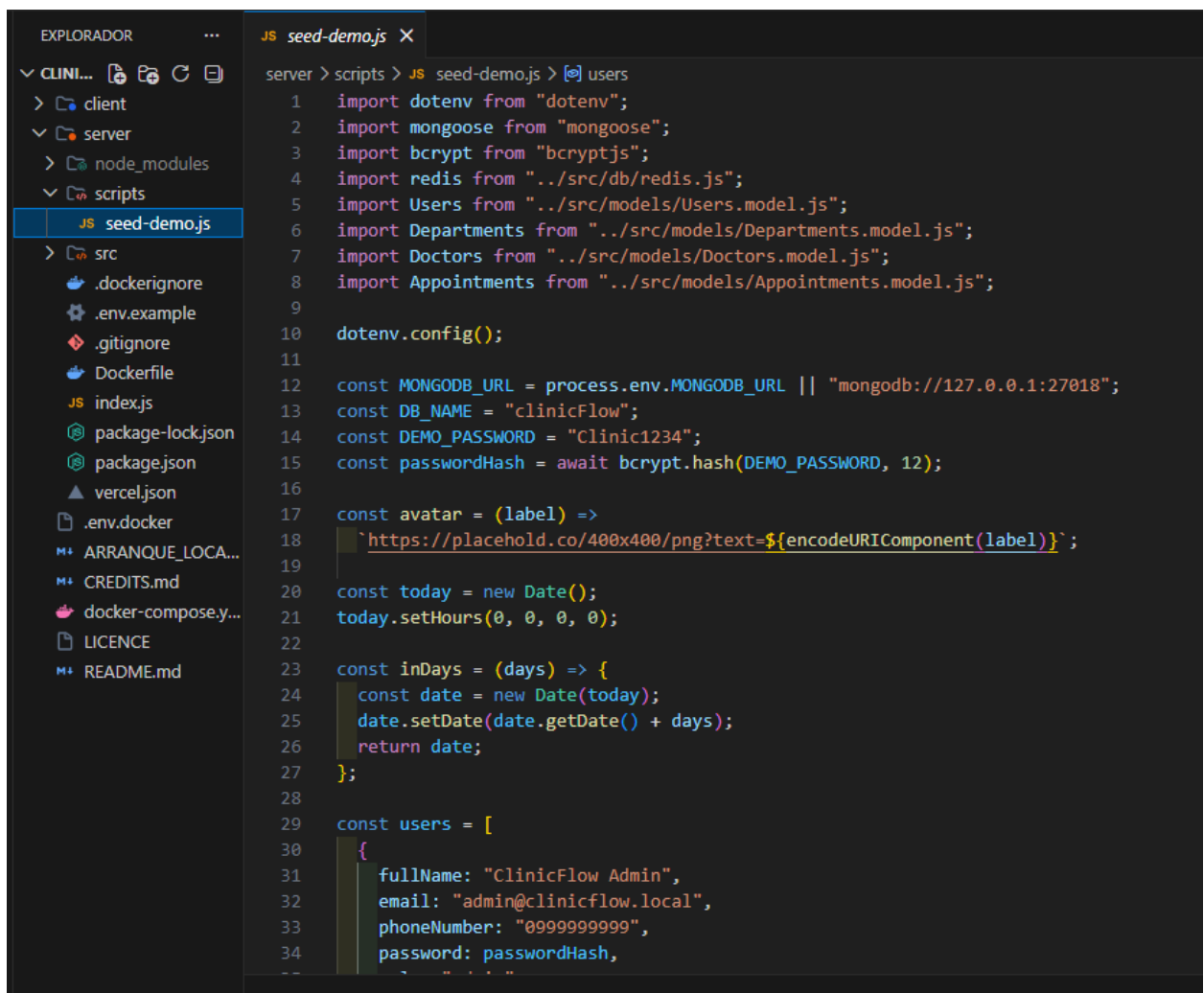
Buenas prácticas y estándares de código aplicados

- Uso de variables de entorno para evitar URLs y credenciales quemadas en el código.
- Centralización de la URL base del backend mediante una función reutilizable.
- Uso de scripts reproducibles para cargar datos de demostración.
- Separación de configuración local, Docker y frontend mediante archivos específicos.
- Manejo de errores en frontend mediante pantalla global y `rejectWithValue` en Redux.
- Invalidación de caché Redis después de operaciones que modifican datos.
- Documentación de comandos de instalación, arranque, seed y verificación del sistema.

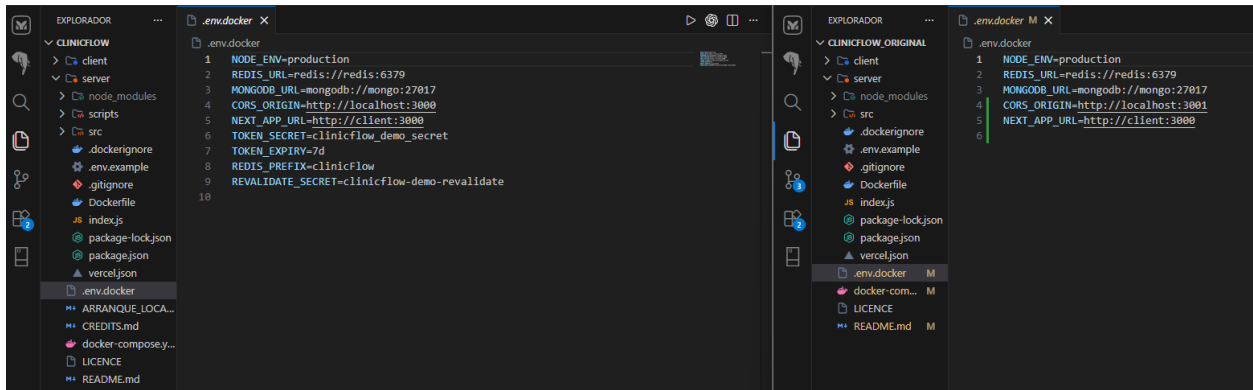
Captura 1. Script de seed agregado



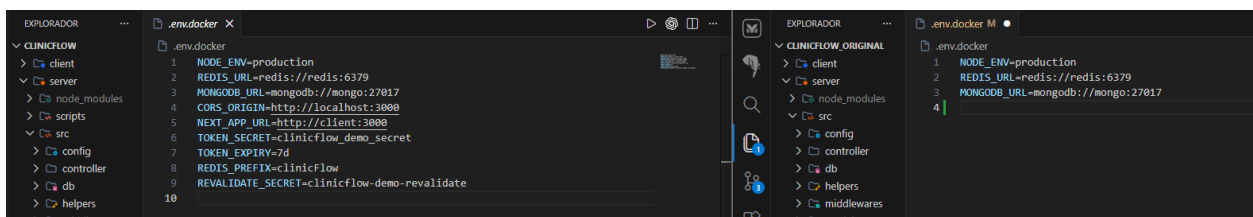
Captura 2. Datos demo cargados desde código



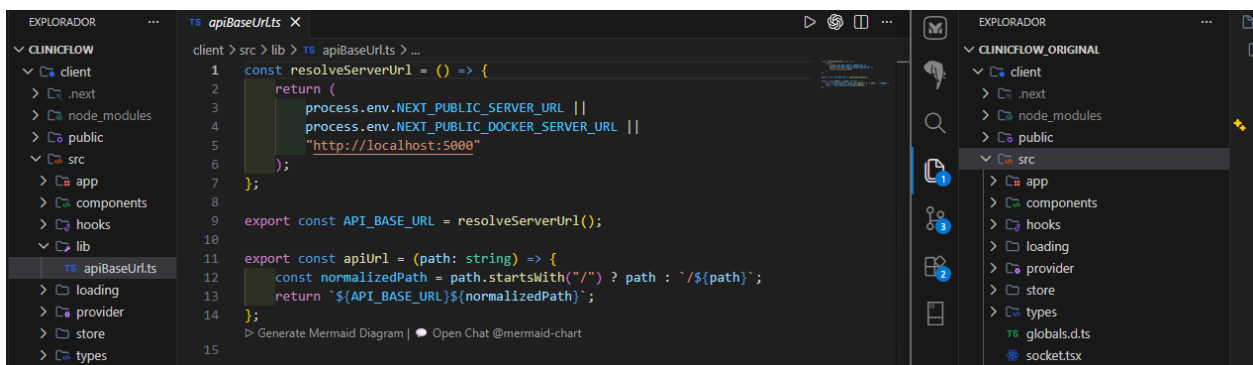
Captura 3. Variables de entorno Docker



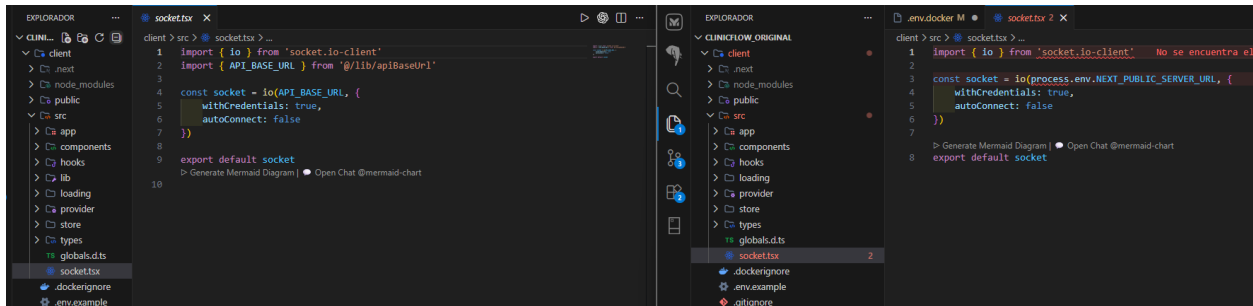
Captura 4. Configuración CORS



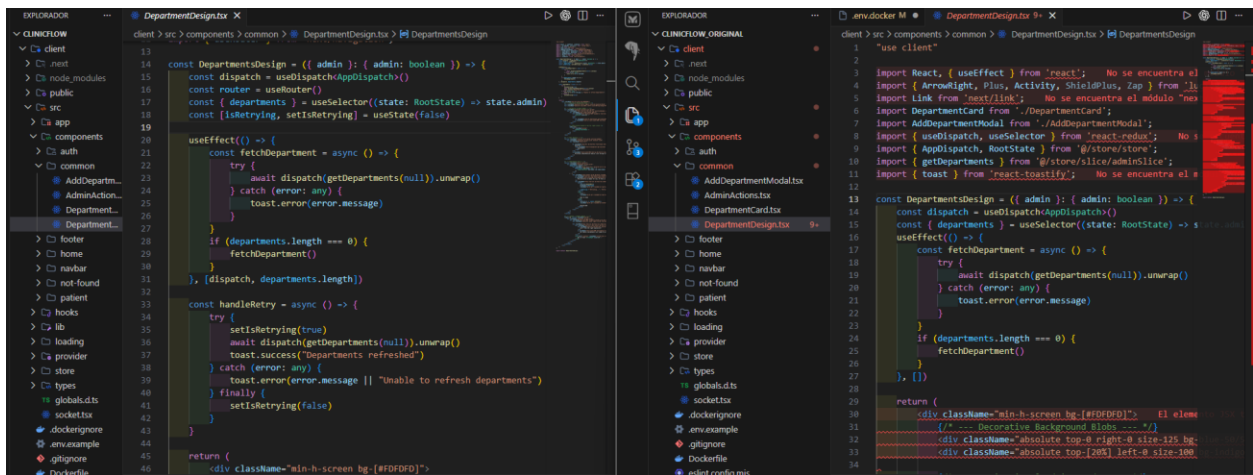
Captura 5. URL base centralizada



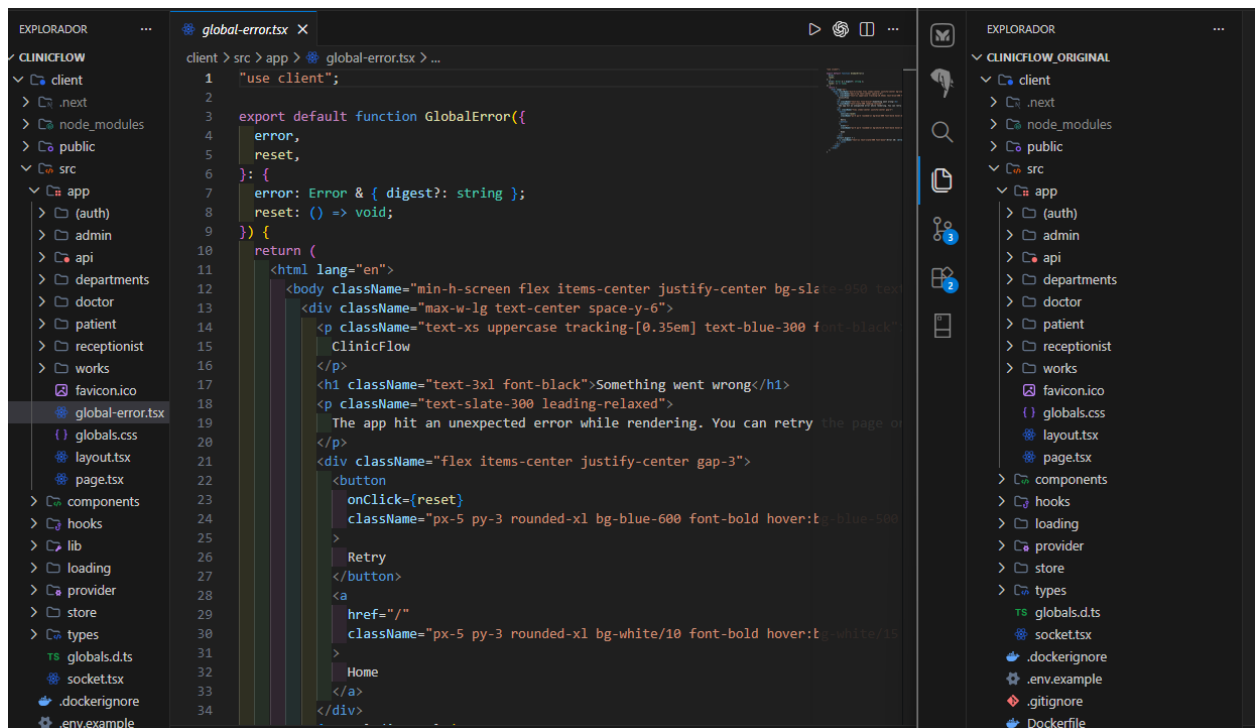
Captura 6. Socket.IO con URL centralizada



Captura 7. Estado vacío mejorado

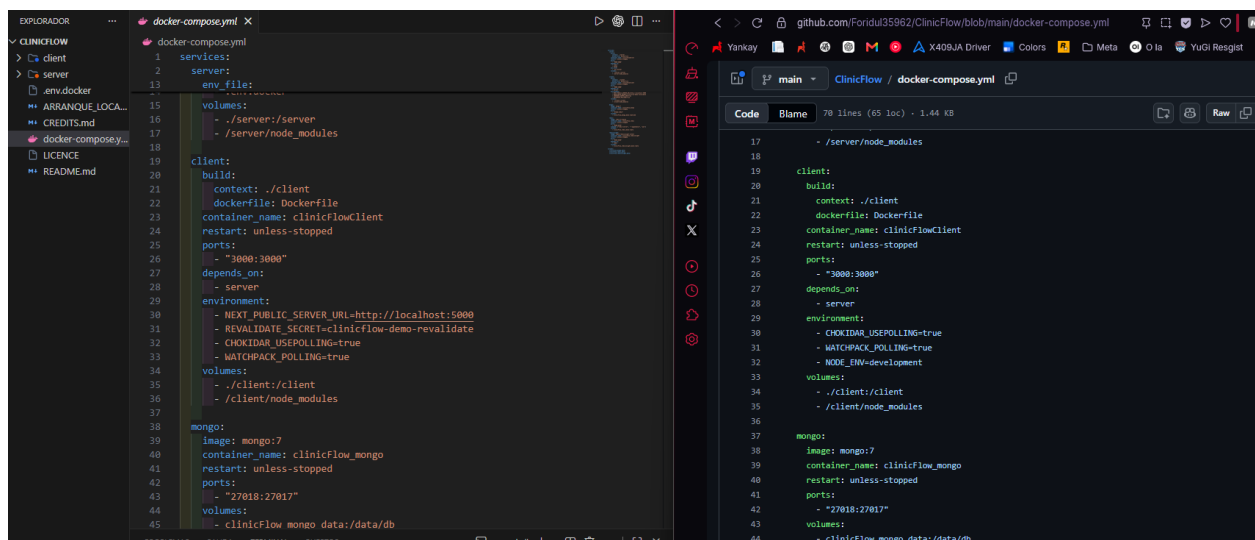


Captura 8. Pantalla global de error



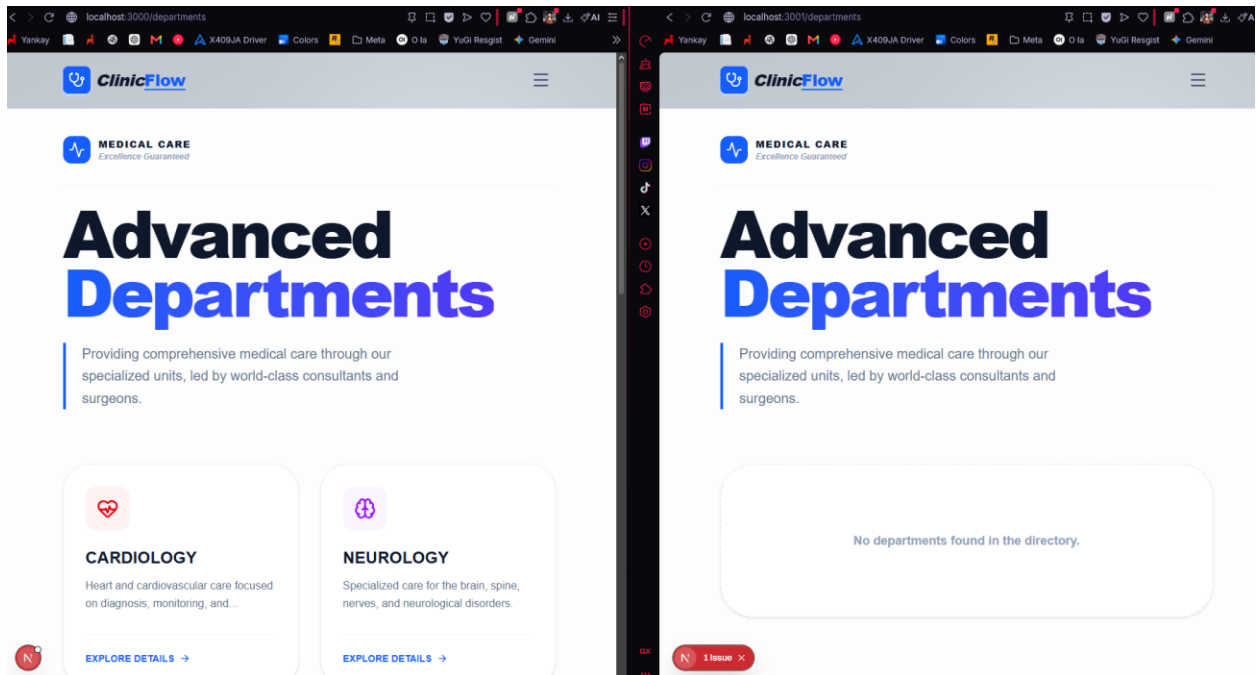
```
1  "use client";
2
3  export default function GlobalError({
4    error,
5    reset,
6  }): {
7    error: Error & { digest?: string };
8    reset: () => void;
9  } {
10   return (
11     <html lang="en">
12       <body className="min-h-screen flex items-center justify-center bg-slate-950 text-white">
13         <div className="max-w-lg text-center space-y-6">
14           <p className="text-xs uppercase tracking-[0.35em] text-blue-300 font-black">ClinicFlow</p>
15           <p></p>
16           <h1 className="text-3xl font-black">Something went wrong</h1>
17           <p className="text-slate-300 leading-relaxed">
18             The app hit an unexpected error while rendering. You can retry the page or
19           </p>
20           <div className="flex items-center justify-center gap-3">
21             <button
22               onClick={reset}
23               className="px-5 py-3 rounded-xl bg-blue-600 font-bold hover:bg-blue-500">
24               Retry
25             </button>
26             <a
27               href="/"
28               className="px-5 py-3 rounded-xl bg-white/10 font-bold hover:bg-white/15">
29               Home
30             </a>
31           </div>
32         </div>
33       </body>
34     </html>
35   );
36 }
```

Captura 9. Docker Compose



```
1  services:
2    server:
3      env_file:
4        - .env.docker
5      volumes:
6        - ./server:/server
7        - ./server/node_modules:/server/node_modules
8    client:
9      build:
10        context: ./client
11        dockerfile: Dockerfile
12      container_name: clinicFlowClient
13      restart: unless-stopped
14      ports:
15        - "3000:3000"
16      depends_on:
17        - server
18      environment:
19        - NEXT_PUBLIC_SERVER_URL=http://localhost:5000
20        - REVALIDATE_SECRET=clinicFlow-demo-revalidate
21        - CHOKIDAR_USEPOLLING=true
22        - WATCHPACK_POLLING=true
23      volumes:
24        - ./client:/client
25        - ./client/node_modules:/client/node_modules
26    mongo:
27      image: mongo:7
28      container_name: clinicFlow_mongo
29      restart: unless-stopped
30      ports:
31        - "27018:27017"
32      volumes:
33        - clinicFlow_mongo_data:/data/db
```

Captura 10. Interfaz con departamentos cargados



6.4. Pruebas

Las pruebas realizadas fueron principalmente manuales, funcionales y de aceptación. Debido a que el proyecto aún no cuenta con una suite automatizada completa, se verificaron los cambios mediante ejecución local, comandos de Docker, navegación en el frontend, revisión de respuestas de API y validación visual de los estados modificados.

Tipo de prueba	Procedimiento	Criterio de aceptación
Prueba de arranque con Docker	<code>docker compose up -d --build</code>	Los contenedores client, server, mongo, redis y redisinsight deben iniciar sin errores críticos.
Prueba de backend activo	Abrir <code>http://localhost:5000</code>	El backend debe responder con el mensaje de servidor activo.
Prueba de frontend activo	Abrir <code>http://localhost:3000</code>	La interfaz debe cargar sin error de renderizado.
Prueba de seed demo	<code>docker compose exec server npm run seed:demo</code>	Debe cargar usuarios, departamentos, doctores y citas de ejemplo.
Prueba de departamentos	Abrir <code>/departments</code>	Deben mostrarse departamentos si el seed fue ejecutado.

Tipo de prueba	Procedimiento	Criterio de aceptación
Prueba de estado vacío	Vaciar departments o no ejecutar seed	Debe mostrarse el estado vacío con botones de acción.
Prueba de CORS/API	DevTools Network: GET /api/user/allDepartment	La petición debe responder 200 sin error CORS.
Prueba de Redux	Editar un doctor o departamento desde admin	La interfaz debe actualizar el elemento modificado sin duplicados ni datos inconsistentes.
Prueba de caché Redis	Modificar datos y volver a consultar	La información debe actualizarse después de invalidar las claves afectadas.

Criterios de aceptación generales

- La aplicación debe iniciar correctamente con Docker Compose.
- El frontend debe consumir el backend sin errores CORS.
- Los datos demo deben cargarse y visualizarse en la interfaz.
- Las variables de entorno deben estar documentadas y alineadas con Docker y ejecución local.
- El estado vacío de departamentos debe orientar al usuario mediante mensajes y acciones visibles.
- Las operaciones que modifican datos deben evitar mostrar caché obsoleta.
- El procedimiento de despliegue debe poder repetirse desde cero en otra máquina.

Observación sobre pruebas automatizadas

Aunque se realizaron pruebas funcionales manuales, todavía queda pendiente implementar pruebas automatizadas para flujos críticos como login, reserva de cita, check-in y dashboard. Esta actividad se considera parte de una futura fase de mantenimiento preventivo.

6.5. Despliegue

El despliegue del sistema modificado se realiza en un entorno local mediante Docker Compose, debido a que esta alternativa levanta de forma integrada el frontend, backend, MongoDB, Redis y RedisInsight. El procedimiento permite validar la aplicación sin instalar manualmente cada servicio externo.

Procedimiento de despliegue con Docker

Paso	Actividad	Comando / verificación
1	Clonar el repositorio	git clone https://github.com/StivenApolo/ClinicFlow_corrected.git

Paso	Actividad	Comando / verificación
2	Ingresar al proyecto	cd ClinicFlow_corrected
3	Revisar variables Docker	Verificar el archivo .env.docker.
4	Construir y levantar servicios	docker compose up -d --build
5	Ejecutar seed demo	docker compose exec server npm run seed:demo
6	Abrir frontend	http://localhost:3000
7	Verificar backend	http://localhost:5000
8	Verificar RedisInsight	http://localhost:5540
9	Detener servicios	docker compose down

Validaciones posteriores al despliegue

- Comprobar que el contenedor del backend se encuentre activo en el puerto 5000.
- Comprobar que el frontend se encuentre activo en el puerto 3000.
- Ejecutar el seed y verificar que se creen registros de demo.
- Abrir la ruta /departments y comprobar que se muestran datos o el estado vacío mejorado.
- Revisar en DevTools que las peticiones al backend no presenten errores CORS.
- Verificar que RedisInsight esté disponible para revisar claves de caché cuando sea necesario.

7. CONCLUSIONES Y RECOMENDACIONES:

Se aplicó el proceso de mantenimiento de software sobre ClinicFlow_corrected, registrando módulos intervenidos, priorizando problemas, documentando cambios, definiendo pruebas y estableciendo un procedimiento de despliegue reproducible. Los cambios más relevantes se relacionan con la configuración de entorno, carga de datos iniciales, corrección de comunicación frontend-backend, mejora del estado vacío y documentación del arranque local.

La incorporación del script seed:demo permite demostrar la aplicación con datos iniciales, lo que corrige el problema de pantallas vacías y facilita la validación funcional. La centralización de la URL del backend y la alineación de variables de entorno reducen errores al pasar entre local, Docker y producción. Además, el estado vacío de departamentos mejora la experiencia del usuario al entregar mensajes y acciones claras.

Como recomendación, se debe continuar con una segunda fase de mantenimiento preventivo orientada a pruebas automatizadas y refactorización de controladores. Esto permitiría reducir el riesgo de regresiones y mejorar la mantenibilidad del backend, especialmente en controladores que aún concentran lógica de validación, acceso a datos, caché y reglas de negocio.

8. BIBLIOGRAFÍA:

- StivenApolo. (2026). ClinicFlow_corrected [Repositorio de GitHub]. https://github.com/StivenApolo/ClinicFlow_corrected
- Foridul35962. ClinicFlow [Repositorio original de GitHub]. <https://github.com/Foridul35962/ClinicFlow>
- ISO/IEC/IEEE. Ingeniería de software: mantenimiento de software. Clasificación de mantenimiento correctivo, adaptativo, perfectivo y preventivo.
- Docker Docs. Docker Compose overview. <https://docs.docker.com/compose/>
- MongoDB Documentation. MongoDB manual. <https://www.mongodb.com/docs/>
- Redis Documentation. Redis data store documentation. <https://redis.io/docs/>